Personal Finance Management App - Complete Requirements Document

For Claude Code Implementation

📋 PROJECT OVERVIEW

Build a comprehensive personal finance management application that allows users to manually track income, investments, and monthly expenses with professional visualizations and analytics. The app should be designed with future bank API integration in mind.

Current Status: Manual data entry only (no bank connections yet) **Future Enhancement:** Framework for Plaid/Open Banking API integration (weekly transaction pulls)

🎯 CORE OBJECTIVES

1. Track income from multiple sources (Salary, Bonus, Freelance, etc.)
2. Manage investment portfolio with real-time P&L calculations
3. Track monthly expenses by category with budget management
4. Visualize financial data with professional charts and analytics
5. Maintain data persistence across browser sessions
6. Prepare architecture for future bank integration
7. Provide mobile-responsive, professional UI/UX
8. Enable data export/import for backups

📱 APP STRUCTURE

Navigation

- **Top Navigation Bar:** Horizontal tab-based navigation with 5 main tabs
- **Tabs:** Dashboard, Income, Investments, Expenses, Settings
- **Mobile:** Responsive navigation (hamburger menu or vertical tabs on screens < 768px)
- **Styling:** Dark theme (dark blue/black background), professional appearance matching reference images

---

🏠 DASHBOARD TAB

Purpose

Overview of total financial health at a glance.

Components

1. Summary Cards (Top Section)

Display 3 large metric cards showing:

Card 1: Net Worth

- Calculation: Total Income - Total Expenses + Investment Values
- Format: $X,XXX,XXX
- Show trend: "↑ +$5,000" or "↓ -$2,000" (30-day change)
- Color: Green if positive, red if negative
- Subtext: "30-day change"

Card 2: Total Monthly Expenses

- Calculation: Sum of all expenses in current month
- Format: $X,XXX.XX
- Show percentage: "23% of income"
- Subtext: "Current month"

Card 3: Investment Portfolio Value

- Calculation: Sum of (quantity × current price) for all investments
- Format: $X,XXX,XXX
- Show P&L: "+$XX,XXX (+12.5%)" in green or red
- Subtext: "Unrealized gains/losses"

2. Net Worth Trend Chart (Main Chart)

- **Type:** Line chart (area chart)
- **X-axis:** Months (last 12 months)
- **Y-axis:** Net worth amount
- **Color:** Gradient teal/cyan
- **Features:**
    - Smooth line showing net worth trajectory
    - Tooltip on hover showing exact amount and date

    - Legend showing "Net Worth" label
    - Responsive sizing
    - Reference lines showing 1-year, 6-month highs
    - Updates automatically when data changes

3. Cash Flow Breakdown Chart

- **Type:** Stacked area chart
- **X-axis:** Months
- **Y-axis:** Amount
- **Lines:**
    - Green: Income
    - Red: Expenses
    - Orange: Net Savings (Income - Expenses)
- **Features:** Tooltips, legend, responsive

4. Recent Transactions List

- **Type:** Simple table showing last 5 transactions
- **Columns:** Date, Type (Income/Investment/Expense), Description, Amount
- **Color Coding:** Green for income, red for expenses, blue for investments
- **Action:** Click to view details or link to relevant tab

5. Financial Health Metrics (Cards)

Small metric cards showing:

- Savings Rate: "(Income - Expenses) / Income × 100%"
- Expense Ratio: "Expenses / Income × 100%"
- Investment Allocation: "Investments / Net Worth × 100%"

💰 INCOME TAB

Purpose

Track all income sources and analyze income patterns.

Components

1. Income Entry Form

Location: Top of tab, in a collapsible card

Fields:

- **Date** (Date picker)
  - Default: Today
  - Required field
  - Format: MM/DD/YYYY display, stored as ISO date
- **Source** (Dropdown)
  - Options: Salary, Bonus, Freelance, Investment Returns, Rental Income, Other
  - Required field
  - Searchable dropdown
- **Amount** (Number input)
  - Required field
  - Validation: Must be > 0
  - Format: Allows decimals ($5,000.50)
  - Error message: "Amount must be greater than 0"
- **Notes** (Text area, optional)
  - Placeholder: "e.g., January salary, Q4 bonus"
  - Max 255 characters
  - Character counter

Buttons:

- **Add Income** (Primary button, only enabled when form valid)
- **Reset Form** (Secondary button)

Validation:

- All required fields must be filled
- Amount must be positive number
- Show inline error messages in red
- Disable "Add Income" button if form invalid
- Clear form after successful submission
- Show success notification: "✓ Income entry added successfully"

2. Income Summary Section

Display above the table:

- **Total This Month:** $X,XXX
- **Total This Year:** $X,XXX
- **Average Income:** $X,XXX
- **Highest Income:** $X,XXX (source name)

3. Income Breakdown by Source (Pie Chart)

- **Type:** Pie chart
- **Data:** All income entries, grouped by source
- **Colors:** Different color for each source
- **Features:**
    - Show percentage on each slice
    - Legend showing source names
    - Tooltip showing: "Source Name: $X,XXX (XX%)"
    - Updates dynamically

4. Monthly Income Trend Chart

- **Type:** Bar chart
- **X-axis:** Months (last 12 months)
- **Y-axis:** Total income that month
- **Colors:** Teal/cyan bars
- **Features:**
    - Tooltip showing exact amount
    - Legend
    - Updates automatically

5. Income List Table

Columns:

Column	Format	Sortable	Notes
Date	MM/DD/YYYY	Yes	Click to sort ascending/descending
Source	Text	Yes	Salary, Bonus, etc.
Amount	$X,XXX.XX	Yes	Sort by amount
Notes	Text (truncated)	No	Show full on hover
Actions	Buttons	No	Edit, Delete

Features:

- Sortable columns (click header to sort)
- Default sort: Date (newest first)
- Show entry count: "Showing 5 of 12 income entries"
- Paginate if > 20 entries (20 per page)

6. Filter Section

Location: Above the table

Filters:

- **Source Filter:** Dropdown (All, Salary, Bonus, Freelance, etc.)
- **Date Range:** From Date - To Date (date pickers)
- **Reset Filters Button**

Behavior:

- Filters apply in real-time
- No need for "Search" button
- Show count of filtered results
- Charts show ALL data (unfiltered)

7. Edit/Delete Functionality

- **Edit Button:** Click to populate form, button changes to "Update"
- **Delete Button:** Shows confirmation dialog "Are you sure? This cannot be undone."
- **Cancel Edit Button:** Returns to "Add" mode
- Success notifications on update/delete

📈 INVESTMENTS TAB

Purpose

Manage and analyze investment portfolio with real-time P&L tracking.

Components

1. Investment Entry Form

Location: Top of tab

Fields:

- **Date** (Date picker)
  - Default: Today
  - Required field
- **Type** (Dropdown)
  - Options: Stock, Crypto, ETF, Bond, Mutual Fund, Other
  - Required field
- **Symbol** (Text input)
  - Examples: AAPL, BTC, VOO, etc.
  - Required field
  - Validation: No spaces, uppercase preferred
- **Quantity** (Number input)
  - Required field
  - Validation: > 0
  - Allow decimals (for fractional shares)
- **Purchase Price** (Number input)
  - Required field
  - Format: Price per unit ($150.25)
  - Validation: > 0
- **Current Price** (Number input)
  - Required field
  - Format: Current price per unit
  - User updates manually (for now, before bank integration)
  - Validation: > 0
- **Notes** (Text area, optional)
  - Placeholder: "e.g., Long-term hold, part of dividend portfolio"

Buttons:

- **Add Investment** (Primary)
- **Reset Form** (Secondary)

Validations: Same as income tab

2. Portfolio Summary Cards

Display prominently:

Card 1: Total Portfolio Value

- Calculation: Sum of (quantity × current price) for all holdings
- Format: $XXX,XXX
- Trend: "↑ +$XX,XXX" (compared to last week or purchase price)

Card 2: Unrealized P&L

- Calculation: Sum of (current value - purchase cost)
- Format: $XX,XXX
- Color: Green if positive, red if negative
- Percentage: "(+12.5%)"

Card 3: Cost Basis

- Calculation: Sum of (quantity × purchase price)
- Format: $XXX,XXX
- Subtext: "Total amount invested"

Card 4: Number of Holdings

- Count of unique investments
- Format: "25 holdings"

3. Asset Allocation Pie Chart

- **Type:** Pie chart
- **Data:** Holdings grouped by Type (Stock, Crypto, ETF, etc.)
- **Show:** Dollar amount and percentage for each type
- **Colors:** Different color per type
- **Legend:** Shows type names and percentages
- **Tooltip:** "Type Name: $X,XXX (XX%)"

4. Holdings Heatmap (Like Reference Image)

- **Type:** Grid/heatmap visualization
- **Display:** Each holding as a colored rectangle
- **Size:** Proportional to current value (larger holdings = larger boxes)
- **Color:**
  - Green shades: Positive P&L (darker = more gains)
  - Red shades: Negative P&L (darker = more losses)
  - Gray: Neutral/break-even
- **Label on each box:** Symbol, percentage of portfolio
- **Tooltip:** "Symbol: Current Value X,XXX, P&L + X (+X%)"
- **Updates:** When prices or holdings change

5. Holdings Table

Columns:

Column	Format	Notes
Symbol	Text	Stock/crypto ticker
Type	Text	Stock, Crypto, ETF, etc.
Quantity	Number	With decimals if needed
Purchase Price	$X.XX	Per unit
Current Price	$X.XX	Per unit (user updates)
Cost Basis	$X,XXX	Quantity × Purchase Price
Current Value	$X,XXX	Quantity × Current Price
P&L $	$X,XXX	Green if +, Red if -
P&L %	X.XX%	Green if +, Red if -
Actions	Buttons	Edit, Delete

Features:

- Sortable by any column
- Default sort: Current Value (highest first)
- Highlight top performers (green) and laggards (red)

- Show total row at bottom summing columns

6. Performance Chart

- **Type:** Bar chart
- **X-axis:** Individual holdings (symbols)
- **Y-axis:** P&L percentage
- **Colors:** Green for positive, red for negative
- **Tooltip:** "AAPL: +12.5% ($1,250)"
- **Sort:** Best to worst performers

7. Filters

- **Type Filter:** (All, Stock, Crypto, ETF, Bond, etc.)
- **Symbol Search:** Text search (real-time)
- **Sort Options:** By P&L, by value, by date added
- **Show count:** "Showing X of Y investments"

8. Price Update Workflow (Current - Manual)

For each investment, user needs to:

1. Click "Edit" on the holding
2. Update "Current Price" field
3. Click "Update"
4. P&L and charts update automatically

Note for future: This will become automatic via Plaid

💳 EXPENSES TAB

Purpose

Track and analyze spending with budget management.

Components

1. Expense Entry Form

Location: Top of tab

Fields:

- **Date** (Date picker)

    - Default: Today
    - Required field
- **Category** (Dropdown)
    - Options: Groceries, Rent, Utilities, Transportation, Entertainment, Dining, Shopping, Personal Care, Office & Equipment, Health, Insurance, Subscriptions, Travel, Gifts, Home/Garden, Streaming/Media, Services, Phone/Internet, Gas/Fuel, Child Care, Clothing, Leisure, Pet Care, Charity, Other
    - Required field
    - Searchable
- **Amount** (Number input)
    - Required field
    - Validation: > 0
- **Description** (Text input)
    - Examples: "Weekly groceries", "Electricity bill"
    - Required field
- **Payment Method** (Dropdown)
    - Options: Cash, Credit Card, Debit Card, Bank Transfer, Check, Other
    - Required field
- **Notes** (Optional text area)

Buttons:

- **Add Expense** (Primary)
- **Reset Form** (Secondary)

2. Budget Tracking Section (Top of Tab)

Overview Card:

- **Total Budget:** $X,XXX (sum of all category budgets)
- **Total Spent This Month:** $X,XXX
- **Remaining Budget:** $X,XXX
- **Progress Bar:** Showing percentage of budget used (green if under, red if over)
- **Status:** "On track" or "XX% over budget"

3. Budget Status by Category

Type: Grid of category cards or table

For each category with budget set:

- **Category Name**
- **Budget Amount:** $XXX
- **Spent This Month:** $XXX
- **Remaining:** $XXX or "OVER by $XXX" (in red)
- **Progress Bar:** Visual indicator (green if under, red if over)
- **Percentage:** "75% of budget" or "125% (25% over)"

Color Coding:

- Green bar: Under budget
- Yellow bar: 75-100% of budget
- Red bar: Over budget

4. Spending by Category Chart (Current Month)

- **Type:** Horizontal bar chart
- **X-axis:** Amount spent
- **Y-axis:** Category names
- **Colors:** Different color per category, or green/red based on budget status
- **Features:**
    - Show actual spent and budget limit as reference line
    - Tooltip: "Groceries: $320 spent of $400 budget (80%)"
    - Update automatically

5. Category Trends Chart (Last 12 Months)

- **Type:** Line chart
- **X-axis:** Months
- **Y-axis:** Amount spent
- **Lines:** Each category is a different colored line
- **Features:**
    - Toggle categories on/off by clicking legend
    - Tooltip showing amount per category per month
    - Identify seasonal spending patterns

6. Spending Heatmap (Calendar View)

- **Type:** Calendar heatmap (like GitHub contribution calendar)
- **Grid:** Each day is a square, each row is a week

- **Color:**
  - Darker shade = more spending that day
  - Light shade = less spending
  - White = no spending
- **Tooltip:** "March 15, 2024: $65.50 spent"
- **Month/Year:** Can navigate to different months

7. Expense Distribution Pie Chart

- **Type:** Pie chart showing all expenses
- **Slices:** Grouped by category
- **Show:** Dollar amount and percentage for each category
- **Colors:** Different color per category
- **Tooltip:** "Groceries: $1,200 (15% of total spending)"

8. Expense List Table

Columns:

Column	Format	Notes
Date	MM/DD/YYYY	Sortable
Category	Text	Sortable
Amount	$X.XX	Sortable
Description	Text	Searchable
Payment	Text	Cash, Card, etc.
Actions	Buttons	Edit, Delete

Features:

- Sortable columns
- Default sort: Date (newest first)
- Paginate if > 30 entries
- Show entry count

9. Filters

- **Category Filter:** Dropdown (All, Groceries, Rent, etc.)

- **Payment Method Filter:** Dropdown
- **Date Range:** From - To date pickers
- **Amount Range:** Min - Max (optional)
- **Description Search:** Text search (searches description field)
- **Reset Filters Button**

Behavior:

- Real-time filtering
- Show filtered count: "Showing 8 of 45 expenses"
- Charts show ALL data (except budget section shows current month)

10. Budget Management

Separate section or Settings Tab:

Budget Setup:

- List of all expense categories
- Input field for each: "Set monthly budget for [Category]"
- Current month spending (read-only)
- Save button
- Validation: Budget must be positive number

Budget Period:

- "This month" = current calendar month (Jan, Feb, etc.)
- Budgets reset automatically on the 1st of each month
- Show: "Budgets reset: April 1, 2024"

⚙️ SETTINGS TAB

Purpose

Manage app settings, data, and configurations.

Components

1. Data Management Section

Export Data:

- **Button:** "Export All Data as JSON"
  - Downloads file: `finance_backup_YYYY-MM-DD.json`

    - Contains: All income, investments, expenses, budgets, metadata
    - Format: Human-readable JSON
    - File size: Display size (e.g., "152 KB")
- **Button:** "Export as CSV"
    - Creates 4 CSV files:
        - `income_YYYY-MM-DD.csv`
        - `investments_YYYY-MM-DD.csv`
        - `expenses_YYYY-MM-DD.csv`
        - `budgets_YYYY-MM-DD.csv`
- **Message:** "Last exported: March 5, 2024 at 2:30 PM"

Import Data:

- **Button:** "Import Data from Backup File"
- **File picker:** Accepts .json files only
- **Preview before import:**
    - "This file contains:"
    - "Income entries: 45"
    - "Investment entries: 12"
    - "Expense entries: 234"
    - "Budget settings: 15"
    - "Import method: ADD to existing data (not replace)"
- **Confirmation dialog:** "Continue? This will ADD to your existing data."
- **Validation:** Check file format, validate data structure
- **Success message:** "✓ Successfully imported 291 entries"
- **Error handling:** "File format is invalid. Please check and try again."

2. Data Statistics

- **Total Transactions:** Income + Investments + Expenses count
- **Date Range:** "Jan 5, 2024 - Mar 5, 2024"
- **Total Data Size:** "2.3 MB"
- **Last Backup:** Date/time of last export
- **Tip:** "Recommendation: Export your data monthly for safety"

3. Data Deletion (Caution Section)

- **Button:** "Clear All Data"
- **Warning Dialog:**

```
⚠️ WARNING: This will permanently DELETE all your data!

This cannot be undone.

Are you sure? Enter "DELETE ALL" to confirm.
[Text input field]
[Cancel] [Confirm]
```

- **Result:** If confirmed, wipes all data from localStorage
- **Recovery:** Only via imported backup file

4. Connected Accounts Section (Placeholder for Future)

Title: "Connected Accounts"

Current State Message:

```
🔗 Bank Integration (Coming Soon)

Connect your bank accounts to automatically import transactions.
This feature will be available in Q2 2024.

[Connect Bank Account Button - DISABLED]

Supported banks: 12,000+ institutions via Plaid
Sync frequency: Weekly
Security: Bank-level encryption
```

Details:

- Explains what will happen when available
- Shows supported features: transaction import, auto-categorization, balance sync
- Placeholder for future connected accounts list

5. Sync Settings (Placeholder)

- **Auto-sync frequency:** "Weekly (when bank integration available)"
- **Auto-categorize:** Toggle (Off) - will categorize new bank transactions
- **Merge settings:** "Automatically merge bank and manual entries"
- **Duplicate detection:** "Enabled - will detect and alert on duplicate transactions"

6. App Settings

Display Preferences:

- **Currency:** Dropdown (USD, EUR, GBP, CAD, AUD, etc.)
  - Affects display format throughout app
  - Default: USD ($)
- **Date Format:** Dropdown (MM/DD/YYYY, DD/MM/YYYY, YYYY-MM-DD)
  - Updates all date displays
  - Default: MM/DD/YYYY
- **Theme:** Radio buttons (Dark, Light)
  - Default: Dark theme
- **Charts Style:** Dropdown (Professional, Colorful, Minimal)

Notifications:

- **Enable notifications:** Toggle
- **Budget alerts:** Toggle - "Notify when category reaches 80% of budget"
- **Overspend alerts:** Toggle - "Alert when budget exceeded"
- **Investment alerts:** Toggle - "Notify of significant P&L changes (>5%)"

7. Help & Support

- **Button:** "View Help Guide"
  - Opens modal with FAQ
  - "How do I add income?"
  - "Where is my data stored?"
  - "Can I recover deleted entries?"
  - "How do bank connections work?"
  - Links to tutorials/video guides
- **Button:** "Contact Support"
  - Email link or support form

8. About Section

- **App Version:** "v1.0.0"
- **Build Date:** Date app was created
- **GitHub Link:** (Optional) Link to source code
- **Privacy Policy:** Link

- **Terms of Service:** Link

🔗 FUTURE: BANK INTEGRATION ARCHITECTURE

Note: These features are NOT implemented yet, but the app should be designed to support them.

Data Source Abstraction

Structure the code so that expenses can come from two sources:

1. **Manual Entry** (current)
2. **Bank API** (future - Plaid)

Transaction Structure

All expenses (manual and bank) should have:

```json
{
  "id": "unique_id",
  "date": "2024-03-15",
  "category": "Groceries",
  "amount": 125.50,
  "description": "Whole Foods",
  "paymentMethod": "Credit Card",
  "dataSource": "Manual" or "Bank",
  "bankTransactionId": "optional_bank_id",
  "autoCategories": "system_calculated_category",
  "mergeStatus": "original" or "duplicate_detected"
}
```

Future Flow (When Ready)

1. User clicks "Connect Bank Account"
2. OAuth authentication with Plaid
3. System fetches last 90 days of transactions
4. Automatically categorizes using Plaid categories or custom rules
5. Detects duplicates (transaction same amount/date/merchant on same day)
6. Merges with manual entries
7. Weekly auto-sync pulls new transactions
8. User can:
    - Adjust auto-categories

- Approve/reject imported transactions
- Split transactions if needed

Code Readiness

- Add comments indicating where API calls will go
- Create helper functions: `categorizeTransaction()`, `detectDuplicates()`, `mergeTransactions()`
- Design data structures to support dual data sources
- Prepare localStorage structure for bank account metadata

💾 DATA PERSISTENCE & STORAGE

localStorage Implementation

- **Keys:**
  - `finance_app_income` - Array of income entries
  - `finance_app_investments` - Array of investment entries
  - `finance_app_expenses` - Array of expense entries
  - `finance_app_budgets` - Object with budget settings
  - `finance_app_settings` - User preferences (currency, theme, etc.)
  - `finance_app_lastBackup` - Timestamp of last export
- **Data Format:** JSON
- **Max Size:** ~5MB (localStorage limit)
- **Auto-save:** On every action (add, edit, delete)
- **Recovery:** Full backup via export/import

Data Structure Examples

Income Entry:

```
json
```

```
{
  "id": "inc_1234567890",
  "date": "2024-03-01",
  "source": "Salary",
  "amount": 5000,
  "notes": "Monthly salary",
  "createdAt": "2024-03-05T10:30:00Z",
  "updatedAt": "2024-03-05T10:30:00Z"
}
```

Investment Entry:

```json
{
  "id": "inv_1234567890",
  "date": "2024-03-01",
  "type": "Stock",
  "symbol": "AAPL",
  "quantity": 10,
  "purchasePrice": 150.00,
  "currentPrice": 175.50,
  "notes": "Apple shares",
  "createdAt": "2024-03-05T10:30:00Z",
  "updatedAt": "2024-03-05T14:22:00Z"
}
```

Expense Entry:

```json
{
  "id": "exp_1234567890",
  "date": "2024-03-05",
  "category": "Groceries",
  "amount": 85.50,
  "description": "Weekly groceries",
  "paymentMethod": "Credit Card",
  "notes": "Whole Foods",
  "createdAt": "2024-03-05T14:15:00Z",
  "updatedAt": "2024-03-05T14:15:00Z"
}
```

Budget Entry:

```json
```

```
{
  "category": "Groceries",
  "monthlyBudget": 400,
  "currency": "USD",
  "createdAt": "2024-03-01T00:00:00Z"
}
```

🎨 UI/UX DESIGN SPECIFICATIONS

Design System

Color Palette:

- **Primary Background:** #0F172A (dark navy)
- **Secondary Background:** #1E293B (slightly lighter navy)
- **Card Background:** #1E2139 (for cards/containers)
- **Positive/Success:** #10B981 (green) - income, gains, under budget
- **Negative/Alert:** #EF4444 (red) - expenses, losses, over budget
- **Neutral:** #64748B (slate gray) - text, borders
- **Accent:** #06B6D4 (cyan) - charts, highlights
- **Text Primary:** #F1F5F9 (light)
- **Text Secondary:** #CBD5E1 (medium light)
- **Border:** #334155 (slate)

Typography:

- **Font Family:** Inter, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif
- **Headings:** Bold, 20-32px
- **Body:** Regular, 14-16px
- **Labels:** Regular, 12-14px
- **Code/Numbers:** Monospace font for currency values

Spacing:

- Use consistent 4px, 8px, 12px, 16px, 24px, 32px increments
- Cards: 16px padding
- Sections: 24px margin between sections
- Inputs: 8px padding

Borders & Radius:

- Border radius: 8px for cards, 6px for inputs
- Border width: 1px
- Border color: #334155

Layout

Desktop (1024px+):

- Cards in 2-3 column grid where appropriate
- Full-width tables
- Charts: Full width but max-width 1200px
- Sidebar optional

Tablet (768px - 1023px):

- Cards in 2 column grid
- Tables adapt
- Stack charts vertically if needed

Mobile (<768px):

- Single column layout
- Full-width cards
- Convert tables to card layout (one entry per card)
- Stack all elements vertically
- Navigation: hamburger menu or vertical tab stack
- Buttons: Full width
- Touch targets: Minimum 44px height

Interactive Elements

Buttons:

- **Primary Button:** Solid cyan/teal background, white text, 44px height minimum
- **Secondary Button:** Outlined style, transparent background, border, 44px height
- **Danger Button:** Red background, white text (for delete)
- **Hover State:** Slight opacity change or shadow increase
- **Disabled State:** 50% opacity, cursor: not-allowed
- **Active State:** Slightly darker shade

Inputs:

- **Border:** 1px solid #334155
- **Focus:** Bright cyan border, shadow glow
- **Error State:** Red border, red error text below
- **Placeholder:** #64748B (medium gray)
- **Height:** 44px minimum

Tables:

- **Header Row:** Background #334155, bold text
- **Row Hover:** Background #1E2139 (slight highlight)
- **Alternating rows:** Subtle alternating background (optional)
- **Sortable column header:** Cursor pointer, shows up/down arrow icon

Modals/Dialogs:

- **Background:** Semi-transparent dark overlay
- **Modal box:** White text on dark background
- **Width:** 90% on mobile, max 600px on desktop
- **Buttons:** Aligned right or full width on mobile

Notifications:

- **Position:** Top-right corner
- **Background:** Green for success, red for error, blue for info
- **Duration:** Auto-dismiss after 3 seconds
- **Icon:** Check mark for success, X for error, i for info
- **Close button:** Manual dismiss option

Form Design

- **Label position:** Above input
- **Label text:** Bold, 12-14px
- **Required indicator:** Red asterisk after label
- **Validation:**
    - Real-time validation (as user types)
    - Red error text below field if invalid
    - Disable submit button if form invalid
- **Success feedback:** Green checkmark next to valid fields

Data Visualization Standards

- **Chart colors:** Use accessible, distinct colors
- **Legend:** Always include, clickable to toggle series
- **Tooltips:** Show on hover, formatted with currency/units
- **Responsive:** Resize with container, no horizontal scroll
- **Loading state:** Show skeleton or spinner while loading data
- **Empty state:** Show "No data available" with helpful message

---

✅ VALIDATION RULES

Income Form

- Date: Required, cannot be in future
- Source: Required, selected from dropdown
- Amount: Required, must be > 0, max 999,999,999
- Notes: Optional, max 255 characters

Investment Form

- Date: Required, cannot be in future
- Type: Required, selected from dropdown
- Symbol: Required, alphanumeric only, 1-10 characters
- Quantity: Required, must be > 0, allow decimals
- Purchase Price: Required, must be > 0, 2 decimal places
- Current Price: Required, must be > 0, 2 decimal places
- Notes: Optional, max 255 characters

Expense Form

- Date: Required, cannot be in future
- Category: Required, selected from dropdown
- Amount: Required, must be > 0
- Description: Required, max 100 characters
- Payment Method: Required, selected from dropdown
- Notes: Optional, max 255 characters

Budget Settings

- Budget Amount: Must be > 0, numeric, allows decimals
- Category: Must be valid expense category

Data Import

- File: Must be .json format
- Content: Must contain valid data structure
- Validation: Check data types, required fields, value ranges

---

📊 CALCULATIONS & FORMULAS

Key Calculations

Net Worth:

```
Net Worth = Sum(Income) - Sum(Expenses) + Sum(Investment Values)
Investment Value = Sum(quantity × current price for each investment)
```

Investment Profit/Loss:

```
P&L $ = (Current Price - Purchase Price) × Quantity
P&L % = (Current Price - Purchase Price) / Purchase Price × 100
```

Monthly Expenses:

```
Monthly Expenses = Sum of expenses where date is in current month
```

Budget Status:

```
Spent This Month = Sum of expenses in current month of selected category
Remaining = Budget Amount - Spent This Month
% Used = (Spent This Month / Budget Amount) × 100
Status = "Under" if % Used < 100, "Over" if > 100
```

Savings Rate:

```
Savings Rate = (Income - Expenses) / Income × 100
```

Expense Ratio:

```
Expense Ratio = Expenses / Income × 100
```

Investment Allocation:

```
Investment Allocation % = Total Investment Value / Net Worth × 100
```

🎯 USER FLOWS

Flow 1: Adding Monthly Income

1. User navigates to Income tab
2. Fills form: Date (today), Source (Salary), Amount (5000)
3. Clicks "Add Income"
4. Form validates and clears
5. Success notification appears
6. Entry appears in table
7. Charts and summary cards update automatically

Flow 2: Tracking Investment Growth

1. User navigates to Investments tab
2. Adds initial purchase: AAPL, 10 shares @ $150 = $1,500
3. Days later, stock price rises to $175
4. User clicks "Edit" on AAPL entry
5. Updates "Current Price" to 175
6. Clicks "Update"
7. P&L calculates: +$250 (+16.7%)
8. Charts show updated allocation and performance

Flow 3: Monthly Expense Tracking & Budget

1. User navigates to Settings, sets Groceries budget to $400
2. Throughout month, adds expenses: $80, $65, $120, $55
3. In Expenses tab, sees Budget section:
    - Budget: $400
    - Spent: $320
    - Remaining: $80

    - Progress bar at 80% (green)
4. User enters another $100 expense
5. Budget section updates:
    - Budget: $400
    - Spent: $420
    - Remaining: -$20
    - Progress bar at 105% (red, shows "Over")

Flow 4: Data Backup & Recovery

1. User navigates to Settings
2. Clicks "Export All Data as JSON"
3. File downloads: finance_backup_2024-03-05.json
4. User stores file safely
5. Later, user clicks "Import Data from Backup"
6. Selects the JSON file
7. Preview shows: "Income: 45, Investments: 12, Expenses: 234, Budgets: 15"
8. Clicks "Import"
9. All data restored

Flow 5: Analysis & Insights

1. Dashboard tab automatically shows:
    - Net worth trend (last 12 months)
    - Cash flow breakdown (income vs expenses vs savings)
    - Recent transactions
2. User can filter by date range to see specific periods
3. Investment tab shows:
    - Asset allocation pie chart
    - Holdings heatmap showing best/worst performers
    - Performance chart
4. Expense tab shows:
    - Spending by category (bar chart)
    - Category trends (line chart)
    - Spending calendar heatmap showing spending patterns

🔧 TECHNICAL SPECIFICATIONS

Technology Stack

- **Framework:** React (functional components with Hooks)
- **Styling:** Tailwind CSS (utility-first, responsive)
- **Charts:** Recharts (React chart library)
- **Date Handling:** JavaScript Date objects, moment.js (optional)
- **Icons:** Lucide React or similar icon library
- **State Management:** React useState/useReducer
- **Storage:** Browser localStorage API
- **File Handling:** Blob API for export/import

Code Requirements

- **Single File:** Entire app in one React component or well-organized structure
- **Functional Components:** Use React Hooks, no class components
- **Comments:** Clear comments explaining complex logic
- **Error Handling:** Try-catch blocks for file operations, graceful error messages
- **Performance:** Debounce filters, memoize calculations, optimize re-renders
- **Accessibility:** ARIA labels, keyboard navigation, color contrast

Browser Support

- Chrome (latest 2 versions)
- Firefox (latest 2 versions)
- Safari (latest 2 versions)
- Edge (latest 2 versions)
- Mobile browsers (iOS Safari, Chrome Mobile)

File Size Targets

- Initial load: < 500KB
- Charts library: Included
- No external API calls (until bank integration)

📋 REFERENCE SCREENSHOTS & DESIGN INSPIRATION

The user has provided 5 reference screenshots from LedgerZero that should inspire the design:

Image 1 (Dashboard Overview):

- Net Worth Composition chart showing asset allocation
- Portfolio Value metric: $883,720
- Unrealized P&L: +$161,440 (+23.14%)
- Holdings Heatmap visualization
- Top Movers section
- Asset Allocation pie chart

Image 2 (Social Media Post):

- Shows a personal finance app built on Claude Code
- Mentions support for:
  - Multiple bank accounts with API
  - Crypto wallets
  - Crypto exchanges
  - Brokerages
  - Automated categorization
  - Budgets
  - Analytics
- Displays multiple dashboard views

Image 3 (Dashboard with Alerts):

- Budget alerts (highlighted in orange/red)
  - "YouTube Premium: $15.99"
  - "Shopping budget exceeded"
  - "Entertainment budget exceeded"
  - "Dining budget exceeded"
- Net Worth card with trend
- Monthly Burn chart
- Cash on Hand metric
- Net Worth trend chart
- Cash Flow chart

Image 4 (Portfolio Analysis):

- Stock Portfolio Value: $824,914
- Down $859 (-0.7%)

- Monthly Returns showing performance by month
- Asset Allocation pie chart
- Dividend Income section
- Realized P&L (2024)
- Holdings Heatmap
- Top Movers section

Image 5 (Spending Analysis):

- Spending Analysis section
- Spending by Category chart
- Budget Tracker with progress circles
- Category Trends over time
- Spending Heatmap calendar view

Design Elements to Replicate:

- Dark theme with teal/cyan accent colors
- Large metric cards with icons
- Combination of charts (pie, bar, line, heatmap)
- Color coding: Green for positive, Red for negative
- Professional typography and spacing
- Responsive layout
- Clean, uncluttered design

🚀 MVP (Minimum Viable Product) FEATURES

Must Have (Phase 1):

- ✅ Dashboard with summary cards
- ✅ Income entry, list, and charts
- ✅ Investment entry, list, P&L calculation, and charts
- ✅ Expense entry, list, and charts
- ✅ Data persistence (localStorage)
- ✅ Basic budget tracking
- ✅ Add/Delete/Edit functionality
- ✅ Responsive design
- ✅ Form validation

Should Have (Phase 2):

- ✅ Export/Import functionality
- ✅ Advanced filtering
- ✅ Category trends
- ✅ Spending heatmap
- ✅ Help/Tutorial section
- ✅ Settings/Preferences

Nice to Have (Phase 3+):

- 📌 Bank integration (future)
- 📌 Recurring expenses
- 📌 Spending goals
- 📌 Tax reports
- 📌 Multi-user support
- 📌 Cloud sync
- 📌 Mobile app (React Native)

⚠️ IMPORTANT NOTES FOR CLAUDE CODE

What NOT to Include (Yet)

- ❌ Real bank API connections
- ❌ Real stock/crypto price feeds
- ❌ User authentication/login system
- ❌ Cloud database
- ❌ Mobile app (web-based only for now)
- ❌ Payment processing

Critical Success Factors

- ✅ All data saves to localStorage automatically
- ✅ Data persists across page refreshes
- ✅ All charts update dynamically when data changes
- ✅ Mobile responsive on phones (375px+)
- ✅ Professional, clean UI matching reference images
- ✅ No console errors

- ✅ Form validation prevents invalid entries
- ✅ Calculations are accurate

Testing Requirements

After implementation, verify:

1. Add 5 income entries - verify dashboard updates
2. Add 3 investments with different types - verify P&L calculations
3. Add 10 expenses across categories - verify budget tracking
4. Refresh page - verify all data persists
5. Edit an investment's current price - verify P&L updates
6. Set budget limits - verify budget tracking works
7. Export data - verify JSON file is valid
8. Import data - verify entries are restored
9. Test on mobile (iPhone size) - verify responsive
10. Check for console errors - should be none

📞 CLARIFICATIONS & ASSUMPTIONS

Assumptions Made

1. **User has basic financial data to track** (income, investments, expenses)
2. **Monthly budgets** reset on the 1st of each month automatically
3. **P&L calculations** based on user-entered current prices (not real-time APIs)
4. **All currency** in single selected unit (e.g., USD)
5. **Data is private** and not shared with others
6. **No user authentication** needed (single-user, browser-based)

Future Clarifications Needed

1. When ready for bank integration, we'll choose Plaid vs Open Banking APIs
2. Tax calculation features (which countries/rules?)
3. Multi-currency support details
4. Multi-user/family sharing features

🎓 BUILDING INSTRUCTIONS FOR CLAUDE CODE

How to Use This Document

1. Copy the entire document
2. Paste it into Claude Code
3. Add this instruction: "Build this app completely according to these specifications. Make it production-ready."
4. Claude Code will generate the complete React application
5. Test against the checklist above
6. Request refinements as needed

Expected Deliverable

- Single React component or well-organized component structure
- All features working as specified
- Professional UI matching reference images
- Fully functional with localStorage persistence
- No external API calls (future-ready architecture)
- Mobile responsive
- Form validation
- Error handling

REVISION HISTORY

Version	Date	Changes
1.0	2024-03-05	Initial comprehensive requirements document

APPENDIX: QUICK FEATURE CHECKLIST

Dashboard Tab

- [ ] Summary cards (Net Worth, Monthly Expenses, Investment Value)
- [ ] Net worth trend chart (12 months)
- [ ] Cash flow breakdown chart
- [ ] Recent transactions list

- [ ] Financial health metrics

Income Tab

- [ ] Income form with validation
- [ ] Income summary
- [ ] Income by source pie chart
- [ ] Monthly trend bar chart
- [ ] Income list table
- [ ] Filters (source, date range)
- [ ] Edit/Delete functionality

Investments Tab

- [ ] Investment form with validation
- [ ] Portfolio summary cards
- [ ] Asset allocation pie chart
- [ ] Holdings heatmap
- [ ] Holdings table with P&L
- [ ] Performance chart
- [ ] Filters (type, symbol)
- [ ] Edit/Delete functionality

Expenses Tab

- [ ] Expense form with validation
- [ ] Budget overview
- [ ] Budget status by category
- [ ] Spending by category chart
- [ ] Category trends chart
- [ ] Spending heatmap
- [ ] Expense distribution pie chart
- [ ] Expense list table
- [ ] Filters (category, date, amount, description)
- [ ] Edit/Delete functionality

Settings Tab

- [ ] Export as JSON
- [ ] Export as CSV
- [ ] Import from backup
- [ ] Data statistics
- [ ] Clear all data (with confirmation)
- [ ] Connected accounts placeholder

- [ ] Display preferences
- [ ] Help/Support
- [ ] About section

General

- [ ] Dark theme styling
- [ ] Responsive design (mobile, tablet, desktop)
- [ ] localStorage persistence
- [ ] Form validation
- [ ] Success/error notifications
- [ ] Currency formatting
- [ ] Date formatting
- [ ] Accessibility (ARIA labels, keyboard navigation)

END OF REQUIREMENTS DOCUMENT

This document contains all specifications, design requirements, calculations, and functionality needed to build the complete personal finance management app. Feed this to Claude Code for implementation.